Naming Bugs

So, what's in a name? A lot, apparently, including potential bugs! In this column, let us look at how errors in naming or spelling can lead to bugs.

Ken Thompson, the co-creator of UNIX, was asked what he would do differently if he had to do it all over again. He replied that he would add the missing 'e' in the *'creat()'* system call! There are many interpretations of this curt answer. One is that UNIX is so perfect that, except for spelling mistakes like *'creat'*, everything else is fine so there is no need to do anything different, even if it were to be done all over again. Whatever the interpretation of the answer might be, UNIX programmers have to 'learn' to misspell 'create' as 'creat' to get their program to work!

Most programmers consider naming variables a trivial issue, and do not give it the attention that it deserves. One reason is that they think variable names are just, well, names, and have no impact on the executable program. However, this notion is wrong. The main reason is that the code is not written for the machine to read—it is for other programmers to read. Further, variable names can cause bugs, and hence they do have an impact on the executable program! I refer to confusing, misleading or just wrong variable names as 'naming bugs'. This is an unusual notion, and I'll explain this with a few examples.

First, let's look at *confusing* names. Consider the example of these three interrupt-related methods in the *Thread* class in Java: *'interrupt()'*, *'interrupted()'* and *'isInterrupted()'*. By just reading these method names, can you tell what these methods mean, and what they do? It's difficult, even if you're an experienced Java programmer. So, here is the explanation. The method *'interrupt()'* interrupts the current thread; *'interrupted()'* checks whether the current thread has been interrupted, while also clearing the interrupted status of the thread! And the method *'isInterrupted()'* just checks whether the current thread has been interrupted (but doesn't touch the interrupted status flag). As you can see, the method name *'interrupted()'* is unintuitive and not self-descriptive—you need to read the documentation to understand what it does. So, it is a naming mistake (or bug). To add to the confusion between these similar-looking methods, *'interrupted()'* is a static method, whereas the other two are instance methods.

Confusing names like these often lead to bugs. For example, if you see a call *'t1.interrupted();'* where *t1* is of type *Thread*, it might be because of a programming mistake: the programmer might have intended to call the instance method *'isInterrupted()'* instead of the *'Thread.interrupted()'* method. This problem is so common that there is even a static analyser rule that checks for this (see the CodePro AnalytiX warning: *'Improper use of Thread.interrupted()'*)! This confusion arises because of two reasons: one is that the names *'interrupted'* and *'isInterrupted'* are confusing, and have different semantics; and the other is that Java allows calling static methods from instance variables.

When method names or API names are similar-looking, or differ only by capitalisation or a single character, it is easy to use them interchangeably, by mistake. The problem is compounded if the functionality is identical or similar. A good example is mistyping *'wait'* instead of *'await'* in the context of multi-threaded programming in Java. Again, this mistake appears to be common, and hence there is a static analyser rule to warn you of this potential bug (see CodePro AnalytiX warning: *"wait()* invoked instead of *await()"*) .

Confusion in the spelling of names can cause bugs. For example, mistyping the name of an overriding method is a very common bug; in our context on naming bugs, this problem can arise because of cultural differences in spelling. For example, consider the method *'finalize()'* in Java. A British or Indian programmer might mistakenly attempt to override the *'finalise'* method instead of *'finalize'* in Java! In this case, the derivation will have both the inherited *'finalize'* method as well as a programmer-provided *'finalise'* method, which